Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Final Year Project

Reinforcement Learning for Hide-and-Seek

Cathal Hardy

G00424572

Supervisor: Brian O'Shea

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025 / 2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Signed: _____Cathal Hardy_____

Date: _____27-04-25_____

Acknowledgements

I would like to thank my project supervisor, Brian O'Shea, for his guidance and support throughout this project.

I would also like to thank my family and friends for their encouragement and patience over the course of my studies, particularly during the final year.

Finally, a thanks to my fellow classmates. The discussions, shared knowledge, and general camaraderie over the past four years made the experience far more enjoyable, and I wish everyone well in their future careers.

Summary

This project implements a competitive two-agent hide-and-seek simulation using deep reinforcement learning. A seeker agent and a hider agent are placed in a 3D arena built in the open-source Godot 4 game engine, where the seeker's objective is to locate and catch the hider before the episode ends, and the hider's objective is to evade capture for as long as possible.

Both agents use Proximal Policy Optimization (PPO), implemented via the Stable-Baselines3 library, and are trained without any hand-coded rules or pre-programmed strategies. Learning is driven entirely through reward signals: the seeker is rewarded for closing the distance to the hider and for successful catches, while the hider receives penalties for being [3] [2]seen or caught. All decision-making is derived from compact observation vectors containing ray-cast distances, velocity, direction, and positional information.

Training followed a structured curriculum. The seeker was first trained in three stages of increasing difficulty: fixed spawn positions, randomised spawns, and finally a walled arena with obstacles. The hider was then trained against the seeker's learned policy. A final dual self-play training phase was conducted, where both agents alternated training rounds against each other's most recent policy.

The Godot game environment and the Python training code communicate via a TCP socket bridge, with JSON-encoded messages passed between them at each training step. This design keeps the simulation and machine learning layers fully decoupled.

Results showed the seeker successfully developing navigation and pursuit behaviour across the training stages, with mean episode length decreasing as the seeker became more effective. The hider demonstrated evasion behaviour after training against the frozen seeker policy, with mean episode lengths approaching the maximum, indicating the hider was surviving for most of the episode.

Reinforcement Learning for Hide-and-Seek

using Curriculum Learning in Godot

Cathal Hardy | BEng (Hons) Software & Electronic Engineering | Supervisor: Brian O'Shea | Atlantic Technological University

Overview

This project builds a hide-and-seek simulation in Godot where two AI agents learn through deep reinforcement learning. A seeker must catch the hider using only sensor observations, while the hider must survive as long as possible.

The seeker is trained progressively through increasing levels of difficulty, starting with fixed spawn points, then randomised positions, then a walled arena with walls and moveable obstacles. Once sufficiently competent, the hider is then trained against the seeker's learned policy, developing evasion strategies through competition.

Both agents use Proximal Policy Optimization (PPO) with no hand-coded rules, developing emergent strategies through experience alone.

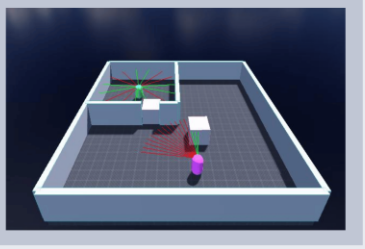
Training Pipeline

Stage 0 Fixed spawn points. Seeker learns basic movement and catching.

Stage 1 Randomised spawn points. Policy generalises across start positions.

Stage 2 Added inner walls and moveable box obstacles. Tests occlusion-aware navigation.

Hider Trained against the pre-trained seeker's learned policy. A 40-step preparation phase gives the hider a head start before the seeker acts.

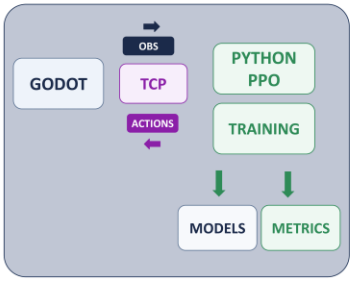


Observation Space (57 features)

Velocity (x, z)	2
Heading (sin θ, cos θ)	2
Carried-box flag	1
Agent position (x, z)	2
Environment raycasts (360°)	16
FOV raycasts (135°)	23
Sees-target flag	1
Target motion features	4
Target position (bearing, dist, abs x/z)	5
Episode progress	1
Total	**57**

System Architecture

Godot and Python communicate via a TCP socket bridge, exchanging observations, actions, and rewards as JSON messages each step.



- Godot sends observations/rewards to Python over TCP; the Python agent returns an action, and the environment steps forward.
- Two parallel environments run simultaneously during seeker training to collect data faster.
- The PPO algorithm updates the policy using rollout batches of 2048 steps.
- TensorBoard tracks episode reward and episode length in real time, making it easy to spot when training stalls or converges.

Technologies & Tools

Godot
3D game engine for the hide-and-seek arena. Provides physics, raycasts, CharacterBody3D agents, and a TCP server node.

GDScript
Godot scripting language. Used for agent logic, reward calculation, environment configuration, and TCP bridge protocol.

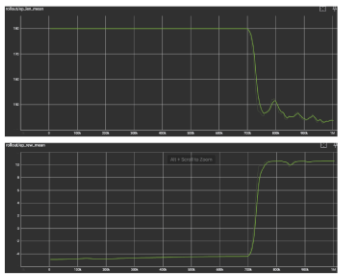
Python
Hosts the GodotEnv wrapper, PPO training scripts, and model loading for inference during evaluation.

Stable-Baselines3
Provides the PPO implementation, SubprocVecEnv, Monitor wrapper, and checkpoint callbacks used throughout training.

TensorBoard
Live view of training metrics to track progress, spot when learning stalls, and compare curriculum stages.

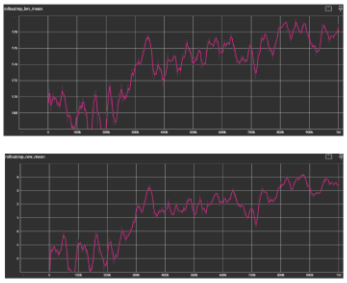
NumPy
Observation and action array handling within the Python environment wrapper and training pipeline.

Results - Seeker



- ep_len_mean stayed near 180 until about 700k steps, meaning most episodes lasted the full length.
- It then dropped to about 143 by 1M steps, showing the seeker was catching the hider faster.
- ep_rew_mean stayed negative until about 700k steps, then rose to around 10, showing clear improvement.

Results - Hider



- The hider showed steady improvement across the full 1M training steps.
- Both ep_rew_mean and ep_len_mean increased consistently from start to finish.
- This suggests the hider got better at staying hidden from the seeker and surviving longer.

Video Demo

Scan to watch the project demo playlist

BEng (Hons) Software & Electronic Engineering | Atlantic Technological University | 2025/2026

Project Poster — Reinforcement Learning for Hide-and-Seek

1 Introduction

Hide-and-seek is a simple game with complex emergent dynamics. When agents must compete, learn to anticipate each other's behaviour, and adapt over time, the problem quickly becomes a compelling benchmark for reinforcement learning. OpenAI's 2019 research demonstrated that agents in a physics-based hide-and-seek environment could develop sophisticated tool use and coordination strategies purely through competition, with no explicit programming of those behaviours.

This project aims to build a scaled-down version of that concept, constrained to run on a standard personal computer, using open-source tooling throughout. The result is a complete end-to-end system: a 3D game environment, a communication bridge between the game and training code, a structured training pipeline, and two trained competing agents.

1.1 Project Goals

- Build a functional 3D hide-and-seek game environment in Godot 4 that can act as a reinforcement learning environment.
- Implement a TCP socket bridge between Godot and Python to allow external control of the simulation.
- Train a seeker agent using a curriculum of increasing difficulty.
- Train a hider agent against the seeker's learned policy.
- Conduct dual self-play training where both agents improve against each other.
- Evaluate and demonstrate agent behaviour in the final trained environment.

1.2 Requirements and Success Criteria

The following measurable success criteria were defined at the outset of the project to evaluate whether each goal had been achieved:

Requirement	Measurable Success Criterion
Godot environment acts as RL environment	Python can connect, reset, and step the environment via TCP without errors
Seeker learns basic pursuit	Mean episode length falls below 100 steps (from 180 max) after Stage 0 training
Seeker generalises across spawn positions	Seeker maintains effective pursuit after Stage 1 randomised-spawn training
Seeker navigates walled arena	Mean episode length stays below 120 steps in Stage 2 despite obstacles
Hider learns evasion	Mean episode length exceeds 150 steps (from 180 max) after hider training
Dual self-play produces improvement	Both agents show measurable reward change across dual training rounds
Training speed is practical	1,000,000 timesteps can be completed overnight on a consumer PC
Observation vector is consistent	Fixed 57-value observation used across all stages without shape errors

Table 1.1 -Project requirements and success criteria

1.3 Scope

The project is a single-agent-versus-single-agent implementation, inspired by but significantly simplified from OpenAI's multi-agent environment. There is one seeker and one hider. Training runs on a standard consumer PC. The environment is a bounded 3D arena with optional inner walls and moveable obstacles. The project does not include the full range of emergent tool use seen in OpenAI's work, as that would require far greater computational resources and a more complex physics environment.

1.4 Report Overview

Section 2 covers background research into deep reinforcement learning and the PPO algorithm. Section 3 describes the overall system architecture. Section 4 details the game environment design. Section 5 explains the training approach and curriculum. Section 6 presents results. Section 7 covers testing. Section 8 discusses problems encountered and how they were resolved. Section 9 concludes the report. Appendix A acknowledges AI tool use during development.

2 Background and Research

2.1 OpenAI Hide-and-Seek

In 2019, OpenAI published research titled “Emergent Tool Use from Multi-Agent Interaction”, in which multiple agents were placed in a physics-based hide-and-seek environment and trained using self-play. The agents had access to moveable boxes and [1]ramps, and were rewarded purely on whether the hiders remained hidden from the seekers.

Over millions of training steps, the agents progressed through distinct emergent phases. Initially, seekers would simply chase hiders. Hiders then learned to push boxes to block doorways. Seekers responded by learning to use ramps to climb over walls. Hiders subsequently learned to move ramps out of reach before the episode began. This cycle of adaptation was driven entirely by competition, with no hand-coded strategies at any stage.

The project documented here takes direct inspiration from OpenAI’s work, though at a significantly reduced scale. The core mechanics, separate reward signals for each agent, preparation phases, and curriculum-based training, are all carried over in simplified form. [1]

2.2 Deep Reinforcement Learning

Reinforcement learning (RL) is a machine learning method in which an agent learns to make decisions by interacting with an environment. At each step, the agent observes the current state, selects an action, and receives a reward signal. Over many episodes, the agent adjusts its behaviour to maximise cumulative reward.

Deep reinforcement learning combines RL with deep neural networks. Instead of storing a lookup table of state-action values, a neural network learns to approximate the policy or value function directly from high-dimensional inputs. This makes DRL practical for problems where the state space is too large to enumerate. [7]

The standard RL loop consists of the following steps:

- The agent observes the environment state.
- The policy maps the observation to an action.
- The environment applies the action and transitions to a new state.
- A reward signal is returned to the agent.
- The agent updates its policy using the reward and new observation.

An episode is one complete run from the initial state until a terminal condition is reached. In this project, an episode ends when the seeker catches the hider or when the maximum step count is reached.

2.3 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy gradient algorithm developed by OpenAI in 2017. It is the primary algorithm used in this project and was also used in OpenAI's hide-and-seek experiments.[2]

PPO is a type of reinforcement learning method where the model is split into two parts. One part, called the policy, decides what action to take in a given situation. The other part estimates how good that situation is, which helps guide learning. These two parts are trained together so the agent can improve its decisions over time.

The key idea behind PPO is that it limits how much the model can change at each step. After each update, it compares the new behaviour to the previous one and restricts large changes. This helps keep training stable and prevents the agent from suddenly performing worse.

PPO is well-suited to this project for several reasons. It works well with both discrete and continuous actions, is widely used in reinforcement learning research, and has a clean open-source implementation in Stable-Baselines3 that integrates easily with a Gymnasium-style environment.

2.4 Stable-Baselines3 and Gymnasium

Stable-Baselines3 (SB3) is a Python library providing reliable, well-tested implementations of reinforcement learning algorithms including PPO. It accepts environments that [3]follow the OpenAI Gymnasium interface, making it straightforward to connect a custom simulation environment to a PPO training loop.

Gymnasium defines a standard structure for RL environments: a 'reset()' method that initialises the episode and returns the first observation, a 'step(action)' method that advances the simulation and returns the next observation, reward, done flag, and info dictionary, and defined observation and action spaces. By wrapping the Godot environment in a Gymnasium-compatible class, SB3 can treat it identically to any other RL environment. [4]

3 System Architecture

3.1 Overview

The system is divided into two main components: the Godot game environment, which runs the simulation, and the Python training stack, which controls the agents and runs the learning algorithm. The two components communicate over a TCP socket connection using JSON-encoded messages.

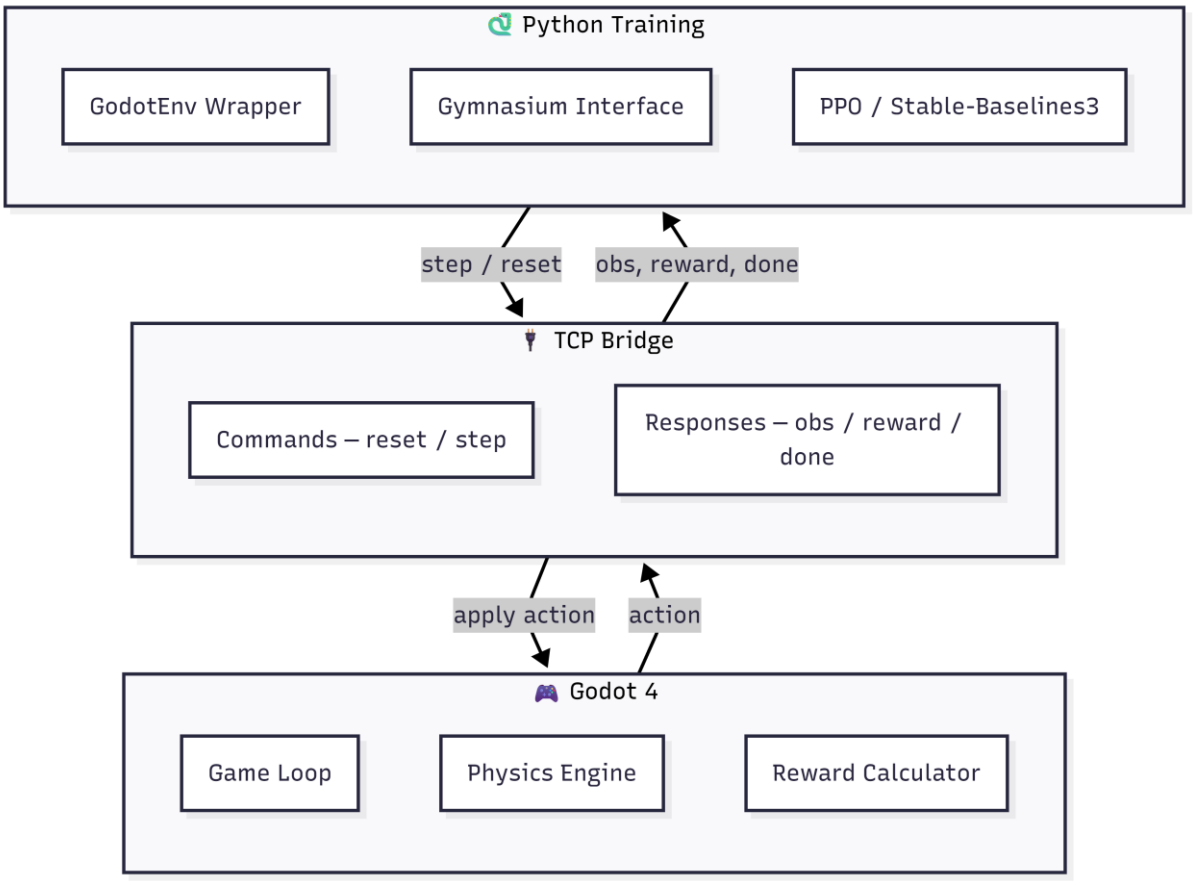


Figure 3.1 - System architecture

3.2 Godot 4 Game Engine

Godot 4 is an open-source game engine with a Python-like scripting language called [5]GDScript. It was chosen for this project for several reasons: prior familiarity with the engine, the similarity of GDScript to Python, good 3D physics support, and the ability to run the exported game headlessly.

The game environment is implemented as a Node3D scene containing the arena, agents, physics objects, and a TCP server node. The main scene script handles all simulation logic: episode resets, action application, reward calculation, and observation generation. A separate bridge script manages all socket communication, keeping the networking code decoupled from the simulation logic.

3.3 TCP Socket Bridge

Communication between Godot and Python is handled by a lightweight TCP socket bridge. Godot runs a TCP server that listens for connections on port 19000 (configurable via command-line argument). When Python connects, it can send two types of command:

- reset -resets the episode and returns the initial observation for both agents.
- step -applies a seeker action and a hider action, advances the simulation by several physics frames, and returns the next observations, rewards, done flag, and info.

All messages are JSON. Godot responds to each command with a JSON object containing an ‘ok’ flag and a ‘result’ payload. The bridge is asynchronous on the Godot side, using GDScript’s ‘await’ keyword to handle the multi-frame step without blocking the game loop.

A ‘bridge_busy’ flag prevents Godot from processing a new command before the previous one has completed, ensuring that the observation returned always corresponds to the action that was sent.

The JSON message format used by the bridge is shown below -a step command sent from Python and the Godot response:

```
# Python sends:
{"cmd": "step", "seeker_action": 1, "hider_action": 3}

# Godot responds:
{"ok": true, "result": {
    "seeker_observation": [...], "hider_observation": [...],
    "seeker_reward": 0.12,  "hider_reward": -0.12,
    "done": false,  "info": {"episode_step": 42, "caught_hider": false}
}}
```

Listing 3.1 -TCP bridge JSON message format

3.4 Python Environment Wrapper

On the Python side, the 'GodotEnv' class wraps the raw socket connection. It handles message serialisation, buffered line reading, and error checking. Built on top of this is 'DualRoleEnv', a Gymnasium-compatible environment class. Depending on whether it is instantiated as 'seeker' or 'hider', it presents the appropriate observation and reward to the training algorithm while using the opponent's frozen policy to generate the other agent's action internally.

This design means that from SB3's perspective, each training run involves a single agent in a standard environment. The complexity of the two-agent interaction is hidden inside the environment wrapper.

The following excerpt from DualRoleEnv.step() shows how the opponent action is generated internally and only the training agent's reward is returned to SB3:

```
def step(self, action: int):
    if self.role == "seeker":
        hider_action, _ = self.opponent_model.predict(
            self._latest_hider_obs, deterministic=True
        )  # frozen opponent acts
        seeker_obs, hider_obs, seeker_reward, _, done, info = \
            self.client.step_both(int(action), int(hider_action))
        return np.asarray(seeker_obs), float(seeker_reward), done, False, info
    else:  # hider role
        seeker_action, _ = self.opponent_model.predict(
            self._latest_seeker_obs, deterministic=True
        )  # frozen opponent acts
        seeker_obs, hider_obs, _, hider_reward, done, info = \
            self.client.step_both(int(seeker_action), int(action))
        return np.asarray(hider_obs), float(hider_reward), done, False, info
```

Listing 3.2 -DualRoleEnv.step() hiding two-agent complexity from SB3

3.5 Technologies Summary

Technology	Role
Godot 4	3D game environment, physics simulation, reward calculation
GDScript	Scripting language for game and bridge logic
Python 3	Training scripts, environment wrappers, model management
Stable-Baselines3	PPO algorithm implementation
Gymnasium	Standard RL environment interface
TCP Sockets	Real-time communication between Godot and Python
TensorBoard	Training metrics visualisation and monitoring

Table 3.1 -Technologies used in the project

4 Game Environment

4.1 Arena Design

The arena is a 20 × 20 metre square bounded by solid outer walls. The wall height, thickness, and arena size are all configurable via exported variables in the main scene script, allowing the environment to be adjusted without recompiling.

Inner walls and obstacle boxes are defined in a GDScript data structure, similar in concept to a JSON array. Each layout entry specifies a list of inner wall segments, each with a position, length, and orientation, along with a list of box positions. At runtime, the main script spawns these geometry elements procedurally. This makes it straightforward to add new layouts or modify existing ones without touching the core simulation logic.

Three layouts are defined: an empty room, a simple room with a single central dividing wall, and a more complex default room with multiple wall segments. The training configuration uses the simple room layout with boxes enabled, providing cover for the hider and navigational challenge for the seeker.

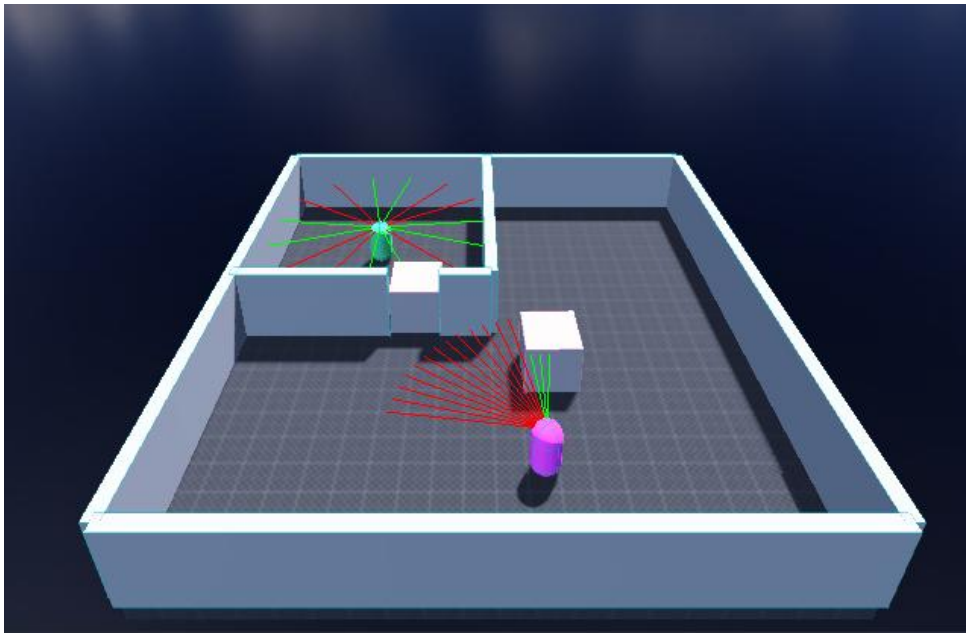


Figure 4.1 - Arena

4.2 Agent Design

Both agents are implemented as CharacterBody3D nodes in Godot, each with a capsule collision shape. They share the same base movement system: five discrete actions (idle, forward, backward, turn left, turn right) applied each step. Movement is applied by setting a target velocity along the agent's local forward axis, with smooth acceleration toward the target velocity using 'move_toward'. Rotation is applied directly as a yaw change per physics frame.

Each agent has two sets of ray casts attached to its head node. The first set is an environment LIDAR: 16 rays evenly distributed across 360 degrees, used to detect walls and obstacles and measure distances. The second set is a vision cone: 23 rays spread across a 135-degree arc in front of the agent, used to detect the opposing agent. The seeker's vision rays detect only the hider, and the hider's vision rays detect only the seeker, using separate physics collision layers.

A preparation phase at the start of each episode keeps the seeker idle for a fixed number of steps, giving the hider time to move into a hiding position before the chase begins. This mirrors the preparation phase used in OpenAI's research.

In the final stage of development, 3D character models were added to both agents to produce a more polished and demonstrable simulation. The models were generated using Rodin AI, a text-to-3D model generation tool, and then rigged and exported through Blender. Animations were sourced from Mixamo: a walk cycle and an idle animation for each agent. The AnimationPlayer node in Godot plays the appropriate animation based on the agent's current action. A coloured point light was also attached to each agent to make them easy to distinguish at a glance -red for the seeker and blue for the hider. Wall surfaces were given a texture to replace the flat grey placeholder material. These changes do not affect agent behaviour or training, but significantly improve the visual quality of the demo.

4.3 Observation Space

Each agent receives a fixed-size observation vector of 57 floating-point values at each step. All values are normalised to the range [-1.0, 1.0]. Using a fixed-size vector is a requirement of PPO, which expects a consistent input shape throughout training.

Index	Count	Description
0–1	2	Velocity: vx, vz (normalised by move speed)
2–3	2	Facing direction: sin(yaw), cos(yaw)
4	1	Carrying box flag (0 or 1)
5–20	16	LIDAR ray distances -360° evenly spaced, detects walls and boxes
21–43	23	Vision ray distances -135° forward cone, detects opponent only
44–45	2	Absolute opponent position: x, z (added in later curriculum stage)
46	1	Opponent detected flag (1 if opponent in vision rays)
47	1	Normalised episode progress: current step / max steps

Table 4.1 -Seeker observation vector (57 values)

The hider uses the same observation structure, with the seeker's position and detection flag in place of the hider's.

The following GDScript from player.gd assembles the observation vector at each step:

```
func get_observation() -> Array[float]:
    var observation: Array[float] = []
    var sees_hider := _can_see_hider()
    observation.append(velocity.x / move_speed)
    observation.append(velocity.z / move_speed)
    observation.append(sin(rotation.y))
    observation.append(cos(rotation.y))
    observation.append(1.0 if carried_box else 0.0)
    observation.append(global_position.x / 10.0)
    observation.append(global_position.z / 10.0)
    observation.append_array(_get_ray_distance_observations(env_rays,
env_ray_length))
    observation.append_array(_get_ray_distance_observations(fov_rays,
fov_ray_length))
    observation.append(1.0 if sees_hider else 0.0)
    observation.append_array(_get_visible_hider_motion_features(sees_hider))
    observation.append_array(_get_visible_hider_position_features(sees_hider))
    return observation
```

Listing 4.1 -Observation vector construction in player.gd

4.4 Action Space

Both agents share the same discrete action space of five actions:

Action ID	Description
0	Idle -no movement
1	Move forward along local forward axis
2	Turn left (rotate positive Y)
3	Turn right (rotate negative Y)
4	Move backward along local forward axis

Table 4.2 -Agent action space

An action repeat of 5 is applied: each action sent by the model is executed for 5 consecutive physics frames before the next observation is generated. This gives each decision a more meaningful physical effect and reduces the total number of model calls per episode.

4.5 Reward Structure

The reward structure is designed to encourage the seeker to close the distance to the hider while discouraging passive behaviour, and to penalise the hider for being caught.

Event	Seeker Reward	Hider Reward
Distance to hider decreases	+0.05 × delta	−0.05 × delta
Hider caught (distance ≤ 2.5m)	+10.0	−10.0
Episode timeout (hider not caught)	−5.0	+5.0
Seeker touches outer wall	Configurable penalty	None

Table 4.3 -Reward structure

The following excerpt from main.gd shows the reward calculation at each step:

```
var distance_delta := max(previous_seeker_hider_distance - current_distance,
0.0)
var seeker_reward := 0.0
var hider_reward  := 0.0

if not in_preparation:
    seeker_reward = 0.05 * distance_delta
    hider_reward  = -seeker_reward
    if caught_hider:
        seeker_reward += catch_reward    # +10.0
        hider_reward  -= catch_reward
    if player.is_touching_outer_wall():
        seeker_reward += outer_wall_penalty

if episode_step >= episode_length_steps and not caught_hider:
    seeker_reward += timeout_penalty     # -5.0
    hider_reward  -= timeout_penalty
```

Listing 4.2 -Reward calculation in main.gd

The distance delta reward provides a dense training signal that guides the seeker toward the hider incrementally, rather than relying solely on the sparse catch reward. The timeout penalty discourages the seeker from adopting passive strategies such as waiting in one location.

5 Training

5.1 Curriculum Learning Approach

Curriculum learning is a training strategy in which an agent is exposed to progressively more difficult tasks over the course of training. Rather than training directly on the full-complexity task, the agent first learns fundamental behaviours in a simpler setting, and those behaviours carry forward as the environment becomes more challenging.

This approach was essential for this project. Attempting to train a seeker in a complex walled arena from scratch led to the agent failing to develop any useful behaviour, as the reward signal was too sparse and the navigation task too difficult. By starting in an open space with a fixed hider spawn, the seeker could learn basic movement and pursuit before being introduced to more challenging conditions.

The training was structured into the following phases:

- Stage 0: Open arena, fixed spawn positions. Seeker learns basic movement and pursuit.
- Stage 1: Open arena, randomised spawn positions. Seeker generalises navigation across the arena.
- Stage 2: Arena with inner walls and obstacles, randomised spawns. Seeker learns to navigate around obstructions.
- Hider Training: Hider trained against the frozen Stage 2 seeker policy.
- Dual Self-Play: Both agents trained alternately against each other's most recent policy.

5.2 PPO Hyperparameters

The following PPO hyperparameters were used throughout training:

Parameter	Value
Policy	MlpPolicy (multi-layer perceptron)
n_steps	2048
batch_size	512
learning_rate	1e-4
gamma	0.99
gae_lambda	0.95
ent_coef	0.01
target_kl	0.01
n_envs	2 (parallel environments, seeker training only)

Table 5.1 -PPO hyperparameters

For the seeker curriculum stages, two parallel Godot environment instances were run simultaneously. SB3 collects rollout data from both environments concurrently, which roughly doubles the rate of experience collection and keeps CPU utilisation close to its maximum. Two instances was the practical limit on my machine before further instances caused CPU saturation and diminishing returns. The hider and dual training phases used a single environment instance, as running two simultaneous Godot windows while also loading the opponent model added too much memory overhead.

5.3 Stage 0: Fixed Spawn Positions

In the first training stage, the seeker and hider were spawned at fixed positions at opposite ends of an empty arena. The hider moved randomly throughout the episode. This gave the seeker a consistent starting scenario to learn basic navigation and pursuit.

Training ran for 100,000 timesteps. By the end of this stage, the seeker had learned to move toward the hider reliably in the open arena, achieving a mean episode length significantly below the maximum of 180 steps.

5.4 Stage 1: Randomised Spawn Positions

In Stage 1, spawn positions were randomised at the start of each episode from a set of predefined positions spread across the arena. A minimum separation distance of 8 metres was enforced to prevent trivial catches. This forced the seeker to develop generalised navigation behaviour rather than learning a fixed route.

Training continued from the Stage 0 model for a further 300,000 timesteps. The seeker maintained effective pursuit across a range of starting configurations.

5.5 Stage 2: Obstacles and Inner Walls

Stage 2 introduced the simple room layout: a single inner wall segment providing partial cover, along with two moveable boxes. Spawn positions continued to be randomised. This required the seeker to navigate around obstacles to reach the hider rather than taking a direct path.

A preparation phase of 40 steps was also introduced at this stage, during which the seeker is forced to idle and the hider can move freely. This more closely mirrors the conditions under which the hider would later be trained.

Training ran for a further 600,000 timesteps from the Stage 1 model, bringing the total seeker training to approximately 1,000,000 timesteps.

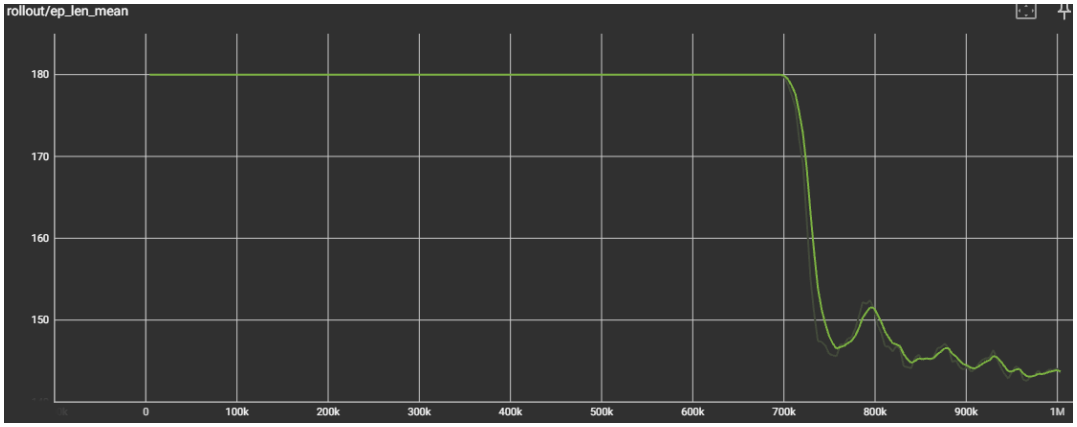


Figure 5.1 - TensorBoard graph of ep_len_mean across seeker training stages

5.6 Hider Training

Once the seeker had completed curriculum training, its policy was frozen and used as a fixed opponent for hider training. The hider's objective is the inverse of the seeker's: to survive as long as possible without being caught.

The hider training script loaded the frozen seeker model and created a Gymnasium environment from the hider's perspective. At each step, the seeker's action was generated internally by querying the frozen seeker model with the latest seeker observation. Only the hider's action was subject to PPO optimisation.

Training ran for 1,000,000 timesteps. The hider developed evasion behaviour, moving away from the seeker and using walls and obstacles for cover. Mean episode length increased toward the maximum of 180 steps, indicating the hider was surviving through most of the episode.

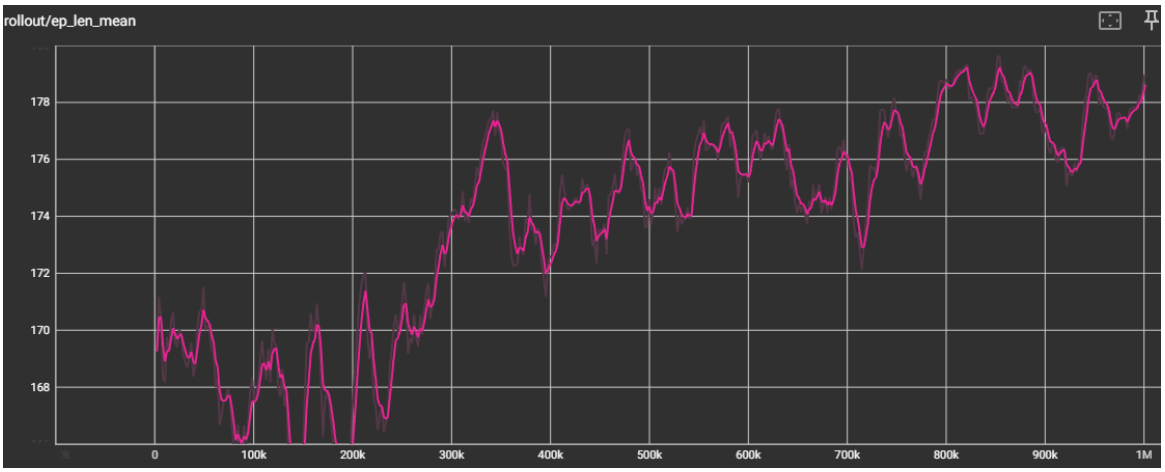


Figure 5.2 - TensorBoard graph of ep_len_mean for hider training

5.7 Dual Self-Play Training

The final training phase used alternating self-play. In each round, the seeker trained for 50,000 steps against the frozen hider policy, then the hider trained for 50,000 steps against the frozen seeker policy. This was repeated for 10 rounds, giving each agent a total of 500,000 additional timesteps and 1,000,000 combined.

The motivation for alternating rather than simultaneous training is stability. If both policies update at the same time, the learning target for each agent shifts unpredictably, which can destabilise training. By freezing one agent while the other learns, each training round has a stable opponent, and the competition ratchets up incrementally.

After each round, the updated model was saved to disk and loaded as the opponent for the following round.

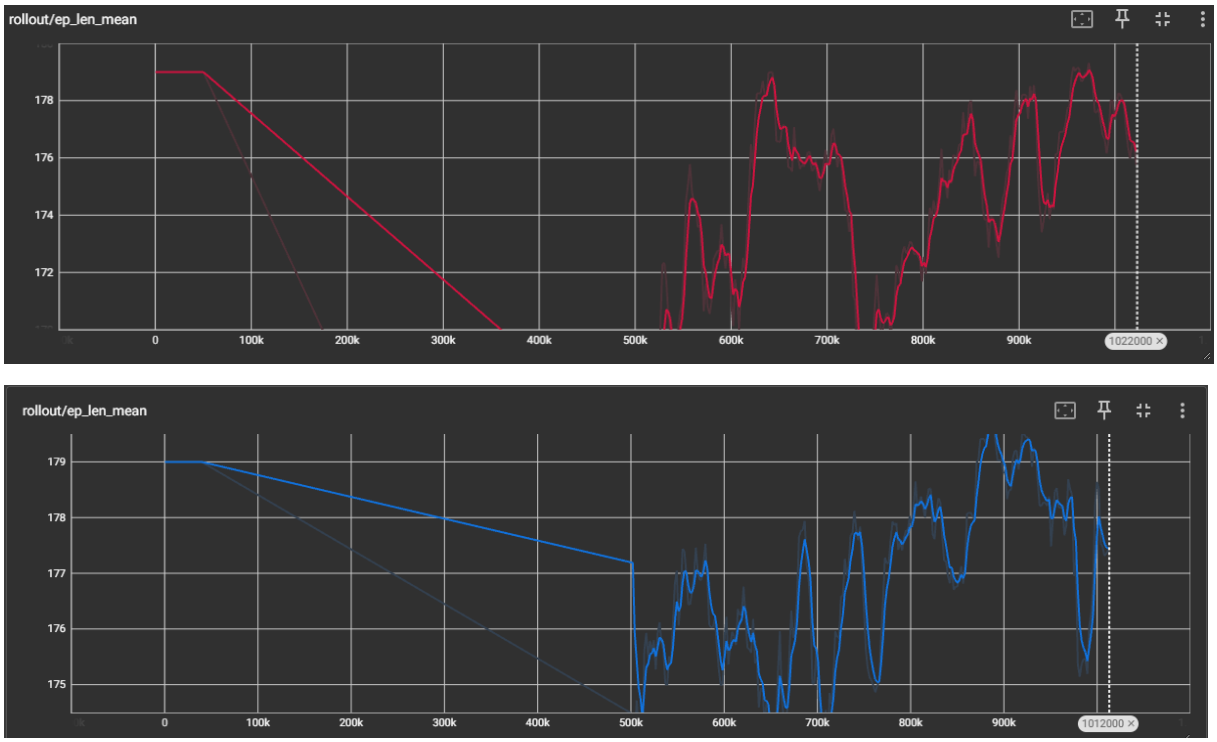


Figure 5.3 - TensorBoard graphs of ep_len_mean for both agents during dual training rounds

Red: seeker, Blue: Hider

6 Results

6.1 Seeker Training Results

Across the three curriculum stages, the seeker demonstrated clear improvement in pursuit behaviour. In Stage 0, mean episode length dropped steadily as the seeker learned to close distance on the hider. In Stage 1, performance was initially lower as the seeker adapted to varied spawn positions, but recovered and generalised well. In Stage 2, the introduction of inner walls caused an initial regression, followed by recovery as the seeker learned to navigate around obstructions.

Key indicators of seeker performance:

- Mean episode length decreasing indicates the seeker is catching the hider more quickly.
- Mean reward increasing reflects both successful catches and consistent distance-closing behaviour.
- The seeker developed a strategy of approaching the hider directly, correcting course when blocked by walls.

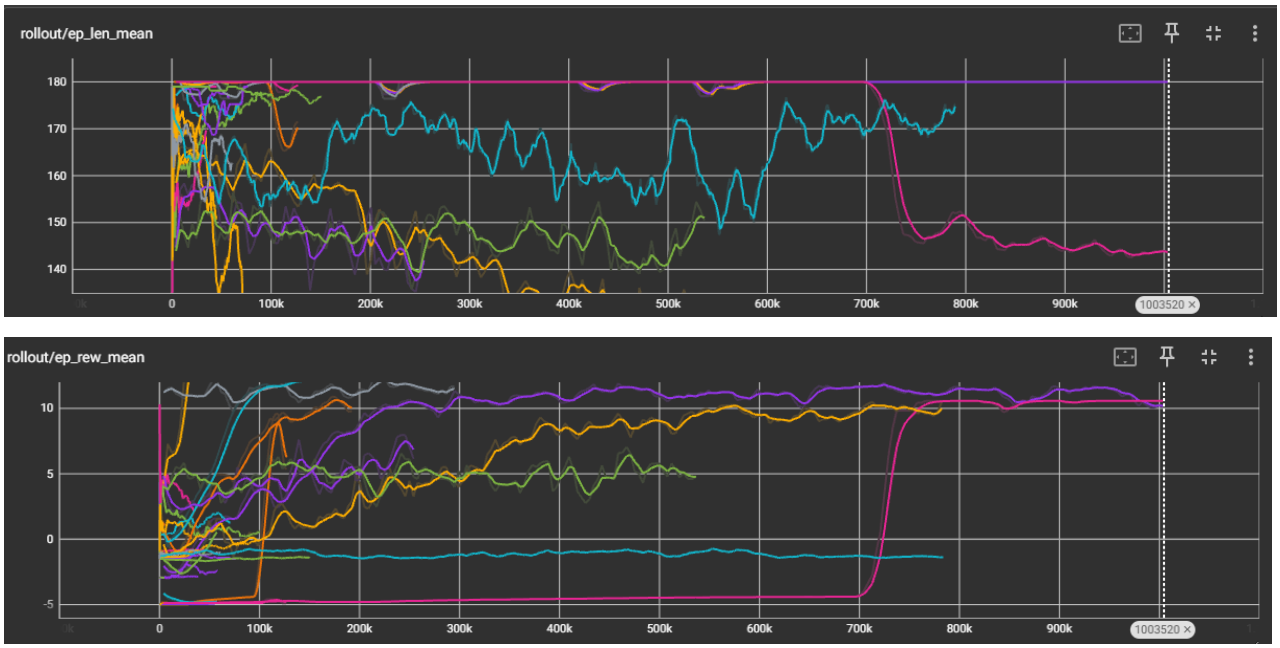


Figure 6.1 - TensorBoard screenshot showing seeker ep_len_mean and ep_rew_mean across full training

6.2 Hider Training Results

The hider training showed a clear improvement in evasion effectiveness. At the start of training, the hider would often be caught within 60–80 steps. After 1,000,000 timesteps of training against the frozen seeker, mean episode length increased toward 180 steps.

The hider developed a strategy of moving away from the seeker, using walls as cover, and avoiding open areas where the seeker's vision rays could detect it easily.

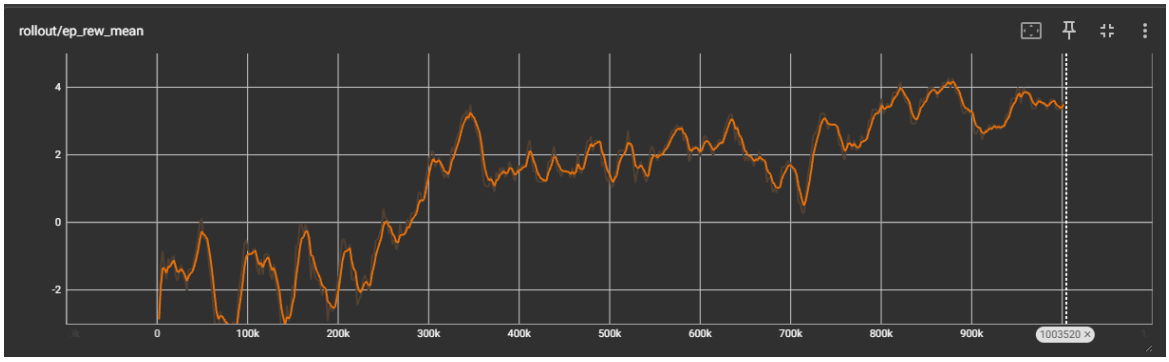


Figure 6.2 - TensorBoard screenshot showing hider ep_len_mean increase during hider training

6.3 Dual Training Results

During dual self-play training, the competitive dynamic between agents was observable. As the seeker's policy improved in a given round, the hider's performance in subsequent rounds would adjust accordingly. The agents continued to adapt to each other across all 10 rounds.

The dual training demonstrated the core concept of the project: two agents improving through competition, without any hand-coded strategies at any stage.

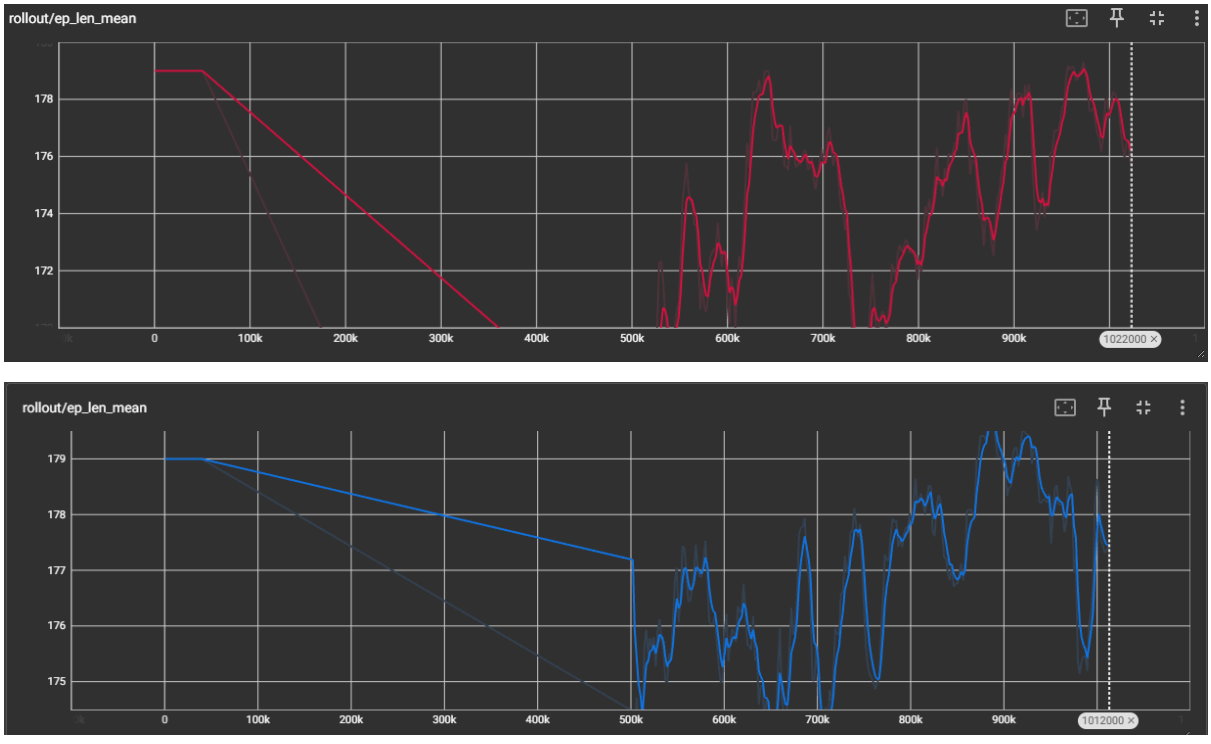


Figure 6.3 - TensorBoard screenshots from dual training showing both agents' metrics

6.4 Observed Agent Behaviour

In demonstration runs using the trained models, the following behaviours were consistently observed:

- The seeker approaches the hider directly when line of sight is available, adjusting heading using both vision ray feedback and positional information.
- When the hider is behind a wall, the seeker navigates around it rather than becoming stuck.
- The hider moves away from the seeker when detected and uses inner walls and boxes as cover.
- During the preparation phase, the hider moves to a covered position before the seeker becomes active.

These behaviours emerged from reward signals alone. No movement strategies were programmed explicitly.

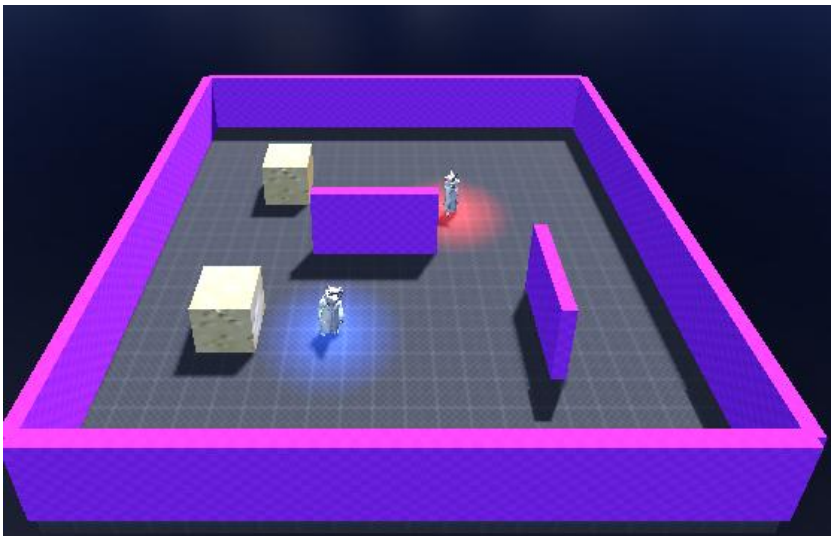


Figure 6.4 – Agents in polished version

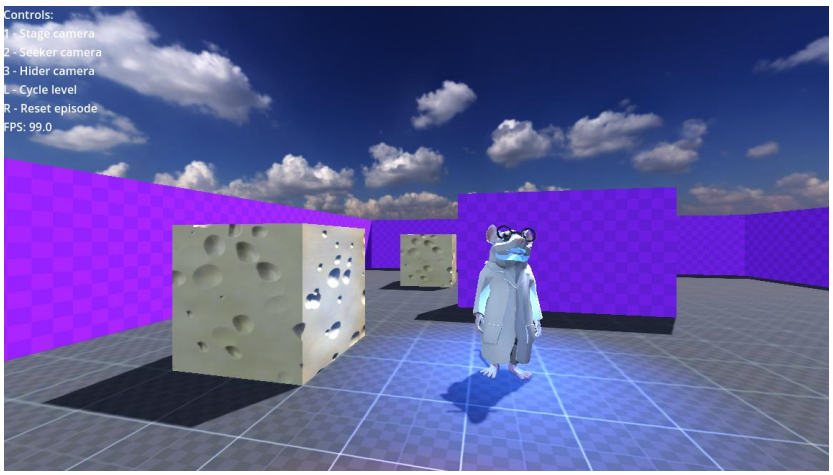


Figure 6.5 – POV of seeker agent

7 Testing

7.1 Test Strategy

A suite of automated unit tests was written using the pytest framework to verify the correctness of the Python environment layer. Testing focused on the two core classes: GodotEnv, which manages the raw TCP socket connection to Godot, and DualRoleEnv, which wraps GodotEnv as a Gymnasium-compatible environment for single-agent training.

All tests are designed to run without a Godot instance. The TCP socket layer is replaced with a Python MagicMock object that returns pre-built JSON response bytes, allowing every code path in the environment to be exercised in isolation. This makes the tests fast, repeatable, and suitable for running in a continuous integration context.

Tests are located in the Python/tests/ directory and can be run with a single command from the project root:

```
pytest Python/tests/ -v
```

7.2 GodotEnv Unit Tests (test_godot_env.py)

Twenty unit tests cover the GodotEnv class. The key areas tested are:

- Connection -verifies that connect() calls socket.connect() with the correct host and port.
- Reset -verifies that reset_both() sends a JSON reset command and correctly parses the seeker and hider observations from the response.
- Step -verifies that step_both() serialises both agent actions into the JSON step command and correctly unpacks the returned observations, rewards, done flag, and info dictionary.
- Single-agent step -verifies that the step() method used during seeker-only or hider-only training correctly dispatches and returns the appropriate agent's data.
- Error handling -verifies that a malformed or truncated JSON response raises a clear exception rather than silently returning incorrect data.
- Context manager -verifies that using GodotEnv as a context manager (with GodotEnv() as env:) correctly closes the socket on exit.
- Done propagation -verifies that a done=true in the Godot response is correctly returned as True in the Python step return value.

7.3 DualRoleEnv Unit Tests (test_dual_env.py)

Ten unit tests cover the DualRoleEnv wrapper and associated utilities. These include:

- Observation space shape -verifies that the env.observation_space matches the expected (57,) shape for both seeker and hider roles.
- Action space -verifies that the env.action_space is Discrete(5) for both roles.
- Reset returns correct shape -verifies that env.reset() returns an observation of the correct numpy shape.
- Step returns correct types -verifies that the step return tuple contains correctly typed values: numpy array, float, bool, and dict.
- Reward routing -verifies that the seeker-role env returns the seeker reward and the hider-role env returns the hider reward from the same raw step response.
- Done propagation -verifies that a done response from Godot propagates correctly through the DualRoleEnv step.

- RandomPolicy -verifies that the built-in random policy used during initial training produces valid action integers within the action space bounds.
- find_resume_path -verifies that the utility function correctly identifies an existing saved model file to resume from.

7.4 Test Results

All 30 tests pass. The test suite runs in under two seconds on a standard machine. No Godot installation is required to run the tests. This confirms that the Python environment layer correctly handles the communication protocol, observation parsing, reward routing, and error conditions across all tested scenarios.

Test File	Tests	Coverage
test_godot_env.py	20	GodotEnv: connect, reset, step, errors, context manager
test_dual_env.py	10	DualRoleEnv, RandomPolicy, resume utilities
Total	30	All pass -no Godot required

Table 7.1 -Automated test suite summary

8 Problem Solving

8.1 Training Speed

The most significant early obstacle was training speed. Initial tests showed that 10 episodes took approximately 2.5 minutes, far too slow to run the thousands of episodes required for meaningful learning. The bottleneck was the Godot physics simulation running at real-time speed while Python waited for each step response.

The solution came from the Godot RL Agents addon, which includes a 'Sync' node that can accelerate the game simulation to a multiple of real-time speed. Setting the speed multiplier to [6]10x reduced training time proportionally. With this in place, 1,000,000 timesteps became achievable in a reasonable overnight run rather than days.

8.2 Reward Shaping Challenges

Several reward configurations were tested before settling on the final structure. Early experiments included a visibility reward that gave the seeker a positive reward simply for having the hider in its vision cone. This led to the seeker learning to face the hider without moving toward it, farming rewards passively.

Removing the visibility reward and replacing it with a distance-delta reward resolved this. By only rewarding the seeker for reducing the distance to the hider, passive behaviour was no longer beneficial. The catch reward of +10 and timeout penalty of -5 provided the terminal incentives.

The outer wall penalty was also introduced after observing that the seeker would frequently hug the arena boundary, as this provided a predictable navigation path. A penalty for wall contact discouraged this and pushed the seeker to move through the centre of the arena.

8.3 Observation Space Refinement

The initial observation vector contained 29 values and relied entirely on ray-cast distances for spatial awareness. While the seeker could learn to pursue the hider in open spaces, it struggled in the walled arena where the hider could be out of view.

Adding absolute hider position to the observation vector (when the hider was detected) significantly improved performance in Stage 2. This gave the seeker a direct spatial reference to navigate toward rather than relying solely on ray-cast feedback. The observation vector was expanded to 57 values at this point.

8.4 Box Interaction

In the initial implementation, agents could push boxes by walking into them. This was unreliable and made it difficult for either agent to use boxes strategically. The interaction was redesigned to use a grab-and-carry mechanic, where an agent can pick up a nearby box, carry it at a fixed offset in front of them, and release it. While held, the box is frozen and its collision is modified to prevent clipping through walls.

This change was inspired by OpenAI's implementation, which also uses a grab action rather than physical pushing.

8.5 Scope Reduction: Box Use and Ramps

The original project plan included training both agents to strategically use the moveable boxes in the environment, and potentially adding ramps as physics objects, mirroring the emergent tool use observed in OpenAI's research. The boxes were already implemented

and functional -agents could pick them up, carry them, and release them at block slots -and the plan was for the hider to learn to barricade doorways and for the seeker to learn to move obstacles out of the way.

However, after completing the seeker curriculum training it became clear that reaching a point where agents would develop meaningful object interaction would require a significantly greater number of training timesteps than were practical on a consumer PC without GPU acceleration. The seeker curriculum alone took approximately 1,000,000 timesteps to produce reliable pursuit behaviour in a walled arena, and that involved no object interaction at all. Teaching agents to reason about and manipulate physics objects as part of a competitive strategy would likely require an order of magnitude more training.

The decision was made to remove strategic box use from the training scope and treat the boxes purely as static cover in the environment. The grab mechanic remains in the codebase and the boxes are still present in the arena, but neither agent is rewarded or penalised for interacting with them during training. This was a pragmatic scope reduction based on the observed training costs, rather than a failure of implementation.

9 Conclusion

This project successfully built a complete reinforcement learning system for a two-agent hide-and-seek game. Starting from an empty Godot scene, the project produced a functioning 3D simulation environment, a real-time Python-Godot communication bridge, a full curriculum training pipeline, and two competing trained agents.

The seeker agent learned to navigate the arena, pursue the hider, and adapt to the presence of inner walls and obstacles through curriculum training. The hider agent, trained against the seeker's policy, developed evasion strategies and learned to use the environment's cover effectively. Both agents improved further during the dual self-play phase, demonstrating the core concept of competitive co-adaptation.

None of the observed behaviours were programmed explicitly. Movement strategies, evasion patterns, and navigation adjustments all emerged from reward signals alone. This validates the reinforcement learning approach for this type of competitive simulation problem.

The project was constrained to run on a standard PC, which limited the number of training timesteps that could be run in a reasonable time. A significant portion of the development effort went into making training fast enough to be practical, primarily through the simulation speed multiplier provided by the Godot RL Agents addon.

Several areas remain for future development. The grab-and-carry mechanic is already implemented and the boxes are present in the arena, but training agents to use them strategically was beyond what was achievable in the available training time on consumer hardware. With access to GPU-accelerated compute and parallel environment instances, enabling and rewarding object interaction would be the most direct extension of the existing system. Adding ramps as physics objects -as in OpenAI's original research -would further increase the potential for emergent strategic behaviour. Increasing the number of agents on each team would bring the project closer to the full multi-agent setting that inspired it.

In the final stages of the project, visual polish was applied to produce a more complete finished product. Character models generated with Rodin AI were rigged in Blender and animated with Mixamo walk and idle cycles. Coloured point lights and a triplanar wall texture were added to improve the visual clarity of the demo environment. These additions do not affect the underlying RL system but contribute to a more professional and demonstrable result.

Overall, the project achieved its goals and produced a working demonstration of competitive multi-agent reinforcement learning in a custom-built environment. The full source code is available on GitHub [8].

10 References

[1] B. Baker, I. Kanitscheider, T. Maroti, Y. Wu, G. Powell, B. McGrew, and I. Mordatch, "Emergent Tool Use From Multi-Agent Autocurricula," OpenAI, 2019. [Online]. Available: https://openai.com/research/emergent-tool-use

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," J. Mach. Learn. Res., vol. 22, no. 268, pp. 1-8, 2021. [Online]. Available: https://jmlr.org/papers/v22/20-1364.html

[4] M. Towers et al., "Gymnasium," Zenodo, 2023. [Online]. Available: https://zenodo.org/record/8127025

[5] Godot Engine, "Godot 4 Documentation," 2024. [Online]. Available: https://docs.godotengine.org/en/stable/

[6] E. Beeching et al., "Godot RL Agents: A toolkit for training reinforcement learning agents in Godot," GitHub, 2023. [Online]. Available: https://github.com/edbeeching/godot_rl_agents

[7] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[8] C. Hardy, "FYP_Project_CH: Hide and Seek Reinforcement Learning w/Godot," GitHub, 2026. [Online]. Available: https://github.com/hardycathal/FYP_Project_CH

Appendix A -AI Use Acknowledgement

In line with the project module guidelines, this appendix documents the use of AI tools during the development of this project.

A.1 Tools Used

Claude (Anthropic) was used as an AI assistant during development. Interactions occurred through the Claude web interface.

A.2 How AI Was Used

- Debugging assistance: AI was occasionally used to help interpret error messages in GDScript and suggest possible causes. These suggestions were reviewed, tested, and applied manually where appropriate.
- Code structure suggestions: AI was consulted for high-level suggestions when organising parts of the codebase, such as separating components into different scripts. All final design decisions and implementations were carried out independently.
- Boilerplate reduction: AI was used to help draft basic structures for test files and repetitive code patterns. These were fully reviewed, adapted, and integrated into the project based on actual requirements.
- Report writing assistance: AI was used to help organise the report structure and ensure that all required sections were included in line with the project rubric.

A.3 What AI Was Not Used For

AI was not used to design the training architecture, select hyperparameters, define the reward structure, or make any of the key technical decisions in the project. The curriculum training approach, the TCP bridge protocol, the observation vector design, and the dual self-play training strategy were all designed and implemented by myself. All code was tested, debugged, and verified by the student before use.